

```

<url-pattern>/PrimeSearcher.do</url-pattern>
</servlet-mapping>
</web-app>

```

The deployment descriptor file *web.xml* is specific to a Web application, and the app server uses it to map servlet names to its classes. Each Web application has its own *web.xml* file. It maps the servlet names to servlet classes—for example, in our application, *PrimeSearcher.do* is mapped to *PrimeSearcher*, and *PrimeSearcher* is mapped to *Controller* class.

The 'view' part of our Web application is a JSP page (*result.jsp*) that generates output for its client. It looks like the following:

```

<%@ page import="java.util.*"%>
<html>
<body>
<h1 align="center">Prime Number generator View</h1>
<p>
<%
    Iterator it= (Iterator)request.getAttribute("primes");
    out.append("primes are:"+<br>);
    while(it.hasNext())
    {
        out.println(""+it.next()+"<br>");
    }
%>
</p>
</body>
</html>

```

This *result.jsp* is internally translated into a servlet by the container. The JSP gets the attribute named *primes* attached to the *HttpRequest* object by the *Controller*. This *primes* attribute is an iterator that contains the prime numbers generated by the model class.

The *Controller* code is shown below:

```

/*****controller.java*****/
import java.io.*;
import java.util.*;

import javax.servlet.*;
import javax.servlet.http.*;

public class Controller extends HttpServlet {

    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        Long num= new Long(req.getParameter("number"));
        Iterator it= model.generatePrime(num.longValue()).iterator();
        req.setAttribute("primes", it);
        RequestDispatcher view= req.getRequestDispatcher("view.jsp");
        view.forward(req, res);
    }

    private Model model = new Model();
}

```

```

}

```

It is a simple servlet class that extends *HttpServlet*. It takes the help of the *Model* class to generate the prime numbers. This *Model* class contains the actual logic to generate primes. The code looks like this:

```

/*****Model.java*****/
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Model {

    List generatePrime(long l)
    {
        long number=2;
        List primes= new ArrayList();
        primes.add(number);
        long sqrt ;
        while(number<l)
        {
            number++;
            if(checkPrime(number)==true)
            {
                primes.add(number);
            }
        }
        return primes;
    }

    boolean checkPrime(long l)
    {
        long sqrt=(long) Math.sqrt(l);
        for (long i = 2; i <= sqrt; i += 1) {
            if (l % i == 0)
                return false;
            else
                ;
        }
        return true;
    }
}

```

This model class contains the actual logic to generate primes, and returns a list of generated prime numbers to the *Controller*.

To understand the entire functionality, assume that when a client requests our Web application to generate primes, then the request will be transferred to the controller class that gets the number entered by the client from the *HttpRequest* object. This number is passed to a model class, which actually generates prime numbers and has the real logic of our application. The model class returns a list that contains prime numbers. The Controller class gets the iterator of this list and attaches this iterator as an attribute to the *HttpRequest* object, and forwards the remaining functionality to the 'view' of our